Critical Infrastructure Security

IY5612 (IY3612/IY4612) — Academic Year 2025/2026 (Unit 6)

Stephen D. Wolthusen
stephen.wolthusen@rhul.ac.uk

Department of Information Security and Communication Technology
Norwegian University of Science and Technology, Norway

and

Department of Information Security
Royal Holloway, University of London, United Kingdom

Unit 6— Control Systems Architecture

Contents

Introduction to Cyber-Physical Systems and Control Systems

Control Systems Architecture

Basic Control Systems
- Introduction
- Control Systems Architecture
- Real-Time Systems

Attacks on Cyber-Physical Systems and Hybrid Attacks

Part II

Control Systems Architecture

Basic Control Systems

This block introduces basic terminology used in control systems

- Basic Control Systems
- Components of Industrial Control Systems
- Real-Time Systems

Recommended Reading (1)

- K. Stoufer, J. Falco, K. Scarfone:
  NIST Special Publication 800-82 Rev. 3: Guide to Industrial Control Systems (ICS) Security
  U.S. National Institute of Standards and Technology (September 2023)

  Chapter 2
- E. Knapp:
  Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and Other Industrial Control Systems
  Elsevier/Syngress (2011)

Basic Control Systems: PLC (1)

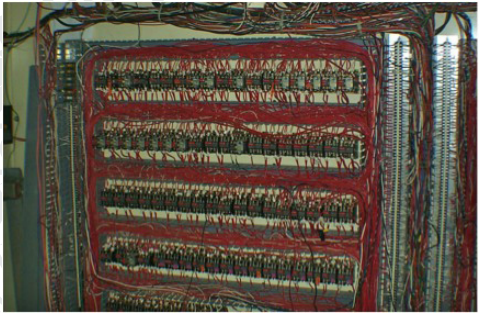
In the simplest case, a control system (or **Process Control System** (PCS) may be at a single site and have only modest complexity (e.g. up to hundreds of input and output variables)

- Historically, and for such simple systems a single **Programmable Logic Controller** (PLC) may be sufficient
- PLCs are specialised computer systems, quite often with integrated I/O facilities beyond what is common in a standard PC architecture
- These systems also may be physically hardened to be placed in semi-hostile environments such as directly on factory floors
- PLCs will normally also require very simple operating environments, mostly executing a single loop

Basic Control Systems: PLC (2)

Programmable Logic Controllers may appear primitive, but compared to the relay logic systems they were replacing, this was a major improvement:

- "Programming" was achieved by sequencing relays and wiring these accordingly
- As with many programs, changes over time may not be fully documented, but are much harder to "debug"



Source: XL Technology

Basic Control Systems: PLC (2)

Programmable Logic Controllers may appear primitive, but compared to the relay logic systems they were replacing, this was a major improvement:

- “Programming” was achieved by sequencing relays and wiring these accordingly
- As with many programs, changes over time may not be fully documented, but are much harder to “debug”

Basic Control Systems: PLC (3)

Such simple (historical) PLCs with very simple, well-defined roles were rarely connected to external systems and hence not subject to external attacks

- Connections to equipment was typically through direct serial lines, often over proprietary protocols
- Interfaces such as EIA-485 can function in electrically noisy environments, but are strictly bus or point-to-point protocols: No routing is possible

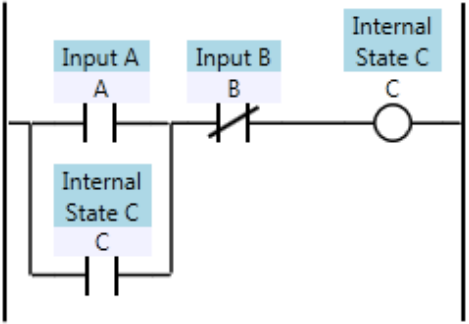
Similarly, the programming environment is deliberatly very simple; many adhering to the IEC61131-3 standard

- Main purpose is to relate input variables to output varialbes, often using a very simple form of ladder logic realising step-by-step changes
- IEC 61499 (function blocks) offers a model for distributed control systems complementary to and layered on IEC 61131

Ladder Logic

Ladder logic simplifies standard propositional logic expressions and represents them as internal state (registers or *coils*) and outputs

- Functions (i.e. predicates) are expressed over inputs and internal state, with each such function called a **rung** of the ladder



Parallel bars (open) indicate a normally open input, crossed bars normally closed (power flows when input is not active; outputs are analogous, represented with circles

Basic RTU Control Systems

Remote Terminal Units (1)

PLC are limited by the range of the connections of the input and output components

- Such connections will permit link lengths of more than 1km, typically sufficing for single sites

However, for more large-scale environments this is insufficient:

- This led to the introduction of **Remote Telemetry Units** (RTU) (also: Remote Terminal Units for more advanced components)

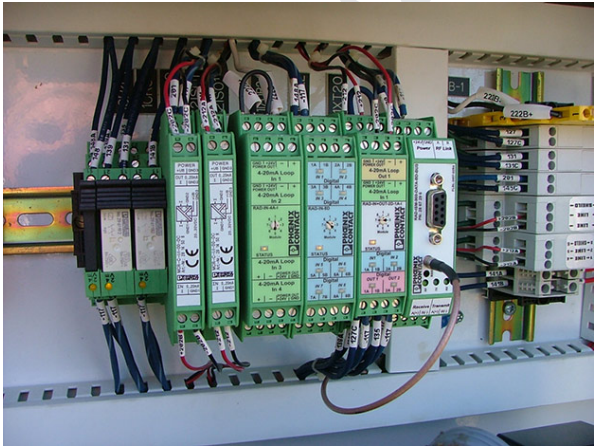
The exact definition and functionality varies and has evolved over time

- Originally simply I/O devices with communication that were not programmable
- Some overlap with PLC has since then occurred, so now RTU can also be deployed as standalone equivalents to PLCs

Basic RTU Control Systems

Remote Terminal Units (2)

A relatively modern RTU combining analog actuator controls, sensors, and communication units:



Source: Schneider Electric

Control System Interfaces — HMI

Human-Machine Interfaces (1)

Control systems will rarely be able to operate completely without changes in configuration or parameters by an operator

- Typically a PLC or RTU will therefore have a **Human-Machine Interface** (HMI)
- Very simple systems will only require minimal interaction such as an operator panel:

Control System Interfaces — HMI

Human-Machine Interfaces (2)

However, providing anything more than minimal commands with a direct interface is both impractical and error-prone

- Commands, programs, and data sets may be more complex and will often require access to data sources
- The HMI will therefore almost inevitably be run on a general-purpose computer

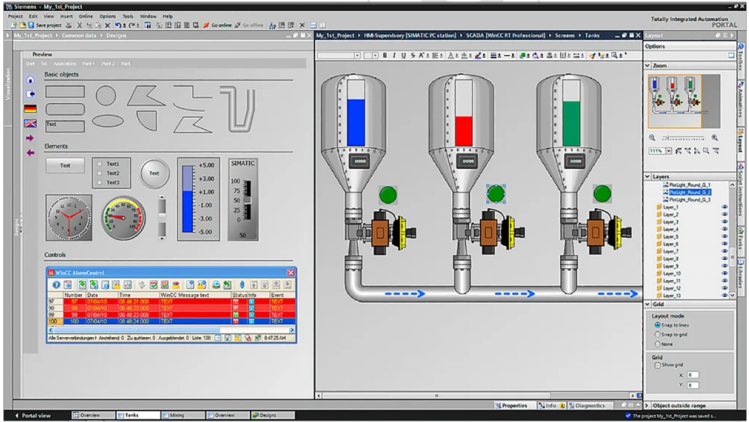
Even where these systems are connected only directly to the control system (PLC), any malware or other attack affecting the PC itself can have an impact on the PLC/RTU and hence the physical system

- Such systems may even be particularly vulnerable: The lifetime of a control system spans multiple PC operating system cycles
- Control system vendors may not qualify newer operating systems or revisions

Control System Interfaces — HMI

Human-Machine Interfaces (3)

An example of a HMI – the Siemens Simatic WinCC/TIA Portal system:

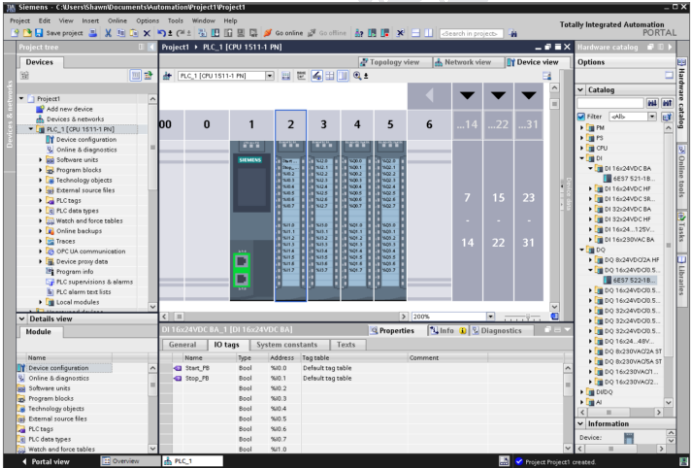


Source: Siemens AG

Control System Interfaces — HMI

Human-Machine Interfaces (3)

An example of a HMI – the Siemens Simatic WinCC/TIA Portal system:



Source: Siemens AG

HMI Functions

More advanced HMI will provide a number of functions:

- Monitoring and changing of PCS parameters
- Display of diagnostic information
- Display of management parameters including maintenance schedules
- Repository for technical information and manuals
- Visualisation of PCS state and performance through schematics and graphs

Common representations during operation include topology diagrams of the PCS, in some cases direct manipulation in the visual interface may also be possible

- This provides the equivalent of direct manipulation from a control room to operators

Network Components (1)

Control System Components

The need or desire for network components as opposed to point-to-point communication arises in a number of areas in control systems:

- Within the control system, the cost of running individual serial lines or buses is significantly greater than a structured network
- While serial buses may cover more than 1km, more complex installations can run against this limit — although serial line extenders exist, they add considerable costs
- More importantly, a strong case exists for exchanging more information with the control system beyond the direct HMI

One common example of such an additional interface is known as a **Data Historian**

The Data Historian

A Data Historian is a database collecting parameters related to operation and performacne

- For this data will generally only need to be collected passively, although this can include a need to interrogate the PLC/RTU about sensor states
- Processes have both cyclical and acyclical properties, which are useful to compare — this role is similar to an intrusion detection system
- Data Historians typically rely on a conventional database back-end; this could be linked to a corporate ERP system: *More on this in the DCS context*
- The main role of a historian is to generate reports and serve as a repository for queries
- Some systems may provide alerts (again, similar to behaviour found in an IDS)

Network Components (2)

Control System Components

Reports and feedback from the PCS and particularly the Data Historian are obviously of considerable interest for operational decisions

- These can be higher-order control systems, but may also reach an organisation's Enterprise Resource Planning (ERP) system
- ERP typically control functions such as whether and when raw material and stock needs to be ordered, or similarly the availability of products
- This is the case for PCS producing discrete results, but even more so for continuously operating products such as the electric grid

Network Components (3)
Control System Components

For interfacing with sensors and actuators, the direct serial links have been increasingly replaced by so-called **field bus** systems since the 1980s

- A large number of competing and specialised systems (e.g. PROFIBUS or Modbus for general PCS, CAN for automotive systems) have been developed
- Only recently has the ISO/IEC attempted to harmonise activities (IEC 61158), but this merely codifies a number of different interfaces

Basic field bus systems are also non-routable but may address thousands of endpoints

- No security measures are present, and only rudimentary integrity protection is in place
- Emphasis is purely on safety and reliability: Redundant cabling, real-time properties

Control System Networks

General-Purpose Network Issues

Almost inevitably, the flexibility and cost advantage of standard (mainly Ethernet and variants, e.g. PROFINET and EtherCAP) networks resulted in their increased use in PCS environments:

- Unlike field bus systems, general Ethernet systems are best effort systems and cannot guarantee well-defined response and cycle times
- Similarly, standard Ethernet networks have a relatively high overhead and are not suitable for the short messages typical of PCS
- As network components may operate in hazardous environments, physical interfaces may also differ from standard networks, including provisions to provide power to sensors and actuators

Note that such networks need **not** be based on TCP/IP!

First Generation SCADA Systems

Simple SCADA Environments

Once the toplogy limitations of field bus systems are removed, it becomes possible to develop more elaborate control system network topologies These are generally referred to as **Supervisory Control and Data Acquisition** (SCADA) systems highlighting the combined roles

- This allows the connection of multiple PLC or RTU units to a shared network, possibly managed by a single HMI or a set of operator terminals each with their own HMI console
- In more complex environments, such systems may also be cascaded or arranged in a hierarchy with a supervisory network interfacing to one or more control system networks and/or field networks

Air Gaps in SCADA Systems (1)

SCADA systems have had safety and reliability as their overriding main focus:

- Security was implicitly provided by having a perimeter within which only trusted operators were assumed
- This is often referred to as an **air gap**, meaning that there is no physical connection between networks (wireless networks are included in this notion)

However, even where no general (e.g. TCP/IP) network access exists, interaction has been possible:

- RTUs must operate over a variety of media including dedicated lines, but also radio (e.g. microwave) links
- For convenience, HMIs and other interfaces have long been provided with dial-up facilities — generally with modest access controls only

Air Gaps in SCADA Systems (2)

Air gaps are increasingly a polite fiction

- The desire to provide information to and from production networks or even to influence (indirectly) control gives a very strong commercial imperative
- Even where air gaps are strictly enforced, e.g. when regulators *require* this unequivocally, this can be subverted

Documented examples exist where modified equipment can provide a RF (e.g. microwave) link either for passive observation or injection of command and control information

- If the supply chain is not secure or physical access not monitored at all time, such links will be difficult to detect or prevent in most control systems environments

Real-Time Requirements (1)

Almost all control systems must operate within well-defined time constraints and are therefore real-time systems

- The exact timing requirements vary — for the purposes of real-time systems speed is not relevant, only that time (anything from nanoseconds to hours in ICS) is critical
- Real-time sytems are characterised by their ability to tolerate missed deadlines with the most demanding being hard real-time systems

Definition (*Hard Real-Time System*)

A system where total correctness depends not only on logical correctness but also the time within the operation is performed; any failure to meet a deadline is considered a total failure

Real-Time Requirements (2)

Many PCS will fall into the hard real-time category, but requirements can also be relaxed in some cases:

Firm Real-Time systems can tolerate missed deadlines for operations, but will suffer some degradation while the utility of a result or operation occurring after the deadline is nil

Soft Real-Time retain some utility of operation results even if deadlines are missed

In addition to the deadline notion, other performance characteristics of real-time systems include bounds on **jitter**:

Definition (*Jitter and Wander*)

The jitter of a signal is defined as the (undesired) deviation from true periodicity of an ideal periodic signal — if frequencies differ more markedlly, this is referred to as a wandering signal

Real-Time Operating Systems (1)

Basic PLC and RTU systems described so far are primitive and typically execute a single loop

- This has the main benefit of allowing a simple calculation for required performance characteristics

Constructing control systems on conventional computer systems, also where additional functionality such as general (e.g. TCP/IP) networking is needed, requires a more complex operating system

- General purpose operating systems such as Microsoft Windows or Linux are **not** real-time systems
- While these are so-called pre-emptive multitasking systems, their design does not provide guaranteed response times
- Background tasks and other operations make determining worst-case execution times for operations very difficult

Real-Time Operating Systems (2)

Specialised **real-time operating systems** (RTOS) will therefore seek to provide some key functions:

Efficiency Most RTOS are so-called microkernel OS, minimising latency

System calls Non-preemptible system calls must be kept brief and deterministic

Scheduling Deterministic scheduling and (hard) prioritisation among tasks

Priority Inversion Systems must control for priority inversion resultig in resource starvation

Memory management Usually no virtual memory is available to avoid paging effects

Real-Time Operating Systems (3)

Examples of RTOS environments:

- **VxWorks** Multi-tasking 64-bit system available on a number of platforms used widely in large-scale systems (e.g. avionics, space systems, industrial systems) but also general computer and telephone network components
- **eCOS** Open-source RTOS with small memory footprint (100k) and networking capabilities, contains outdated TCP/IP stack
- **RTLinux** Hard microkernel RTOS running Linux as a process to keep compatibility, open source
- **QNX** Hard RTOS with Unix-derived interface, frequently used in automotive environments but also e.g. Cisco IOS-XR
- **pSOS** Legacy system used mainly in smaller, lower-performance systems

Real-Time Constraints

Not only operating systems must be chosen carefully if real-time constraints are to be met, even the choice of programming languages may be limited

- Modern programming languages such as Java or C# rely on garbage collection
- Garbage collection automatically manages an application's memory and will "clean up" memory periodically or if it becomes exhausted
- This behaviour may violate real-time constraints and must hence be tightly controlled or avoided overall

Real-Time Constraints

Not only operating systems must be chosen carefully if real-time constraints are to be met, even the choice of programming languages may be limited

- Modern programming languages such as Java or C# rely on garbage collection
- Garbage collection automatically manages an application's memory and will "clean up" memory periodically or if it becomes exhausted
- This behaviour may violate real-time constraints and must hence be tightly controlled or avoided overall

Real-time behaviour is easily violated — even security-related patches may obviously cause harm unless they are carefully qualified

Summary

This section provided material on

- Basic Control Systems
- Components of Industrial Control Systems
- Real-Time Systems